

Taller 2 de Inteligencia Artificial

Uso de algoritmos de busqueda y optimizacion por colonia de hormigas

Grupo 5

Nancy Altamirano | Gustavo Berru | Raquel Pacheco | Kevin Viteri

Asignatura	MSDS 6004 Inteligencia Artificial
Alcance	Parte 1 y Parte 2.
Entregable	Informe tecnico con explicacion general del codigo, metricas, resultados e imagenes.

Este documento resume la solucion desarrollada para el Taller 2, explicando la estructura del codigo, el lugar exacto donde el laberinto se representa como grafo, las metricas usadas para comparar algoritmos y las respuestas solicitadas sobre Ant Colony Optimization (ACO).

Contenido

1. Resumen ejecutivo
2. Parte 1 - Uso de algoritmos de busqueda
3. Metricas y comparacion de resultados
4. Imagenes de solucion de los laberintos
5. Parte 2 - Optimizacion por colonia de hormigas
6. Random Search, Grid Search y aplicacion a TSP
7. Conclusiones

1. Resumen ejecutivo

La solución implementada cubre dos bloques del taller. En la Parte 1 se leen laberintos desde archivos de texto, se transforman en grafos no ponderados y se aplican tres algoritmos de búsqueda: BFS, DFS y A*. Aunque el enunciado solicita comparar al menos dos algoritmos, se agregó A* para obtener una comparación más completa entre búsqueda ciega y búsqueda informada.

En la Parte 2 se ejecuta y corrige una implementación de Ant Colony Optimization. La corrección principal consiste en no escoger como mejor ruta cualquier camino corto, sino solo los caminos completos que llegan a la meta. Luego se selecciona el de menor costo y se refuerza con feromona. Adicionalmente se implementa Random Search para probar configuraciones de hiperparámetros.

2. Parte 1 - Uso de algoritmos de búsqueda

2.1 Explicación general del código

La solución de la Parte 1 está organizada en tres archivos principales. **P1_util.py** contiene la función de colores para representar paredes, espacios libres, entrada y salida. **P1_MazeLoader.py** se encarga de cargar el archivo .txt, normalizar el laberinto, convertirlo en grafo y dibujar los resultados. **P1.py** contiene los algoritmos BFS, DFS y A*, la reconstrucción de rutas, el cálculo de métricas, la generación de imágenes y la exportación de resultados en CSV.

La ejecución inicia leyendo cada archivo de laberinto. Luego se obtiene el grafo, se identifican la entrada **E** y la salida **S**, y se aplican los algoritmos. Cada resultado se guarda con la ruta encontrada, el orden de nodos visitados, los nodos expandidos, el tamaño máximo de la frontera y el tiempo de ejecución. Finalmente, el método **plot_maze()** genera una imagen del laberinto con las celdas visitadas y la ruta final.

2.2 ¿En qué parte se representó el laberinto como grafo?

El laberinto se representa como grafo en el archivo **P1_MazeLoader.py**, específicamente en la función **get_graph()**. En esta función, cada celda transitables del laberinto se convierte en un nodo. Las celdas transitables son los espacios en blanco, la entrada **E** y la salida **S**. Las paredes marcadas con **#** no se incluyen como nodos.

Después de crear los nodos, la función revisa los vecinos de cada celda en cuatro direcciones: arriba, abajo, izquierda y derecha. Si el vecino también es transitable, se agrega una arista en la lista de adyacencia. Por eso, el grafo resultante es un grafo no ponderado donde cada movimiento entre celdas vecinas tiene costo uniforme.

Fragmento clave de la representación como grafo:

```
PASSABLE_CELLS = {' ', 'E', 'S'}

for row_index, row in enumerate(maze):
    for col_index, cell in enumerate(row):
        if cell in PASSABLE_CELLS:
            node = (row_index, col_index)
            graph[node] = []
            if cell == 'E':
                self.start = node
            elif cell == 'S':
                self.end = node

directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
for row, col in graph:
    for d_row, d_col in directions:
        neighbor = (row + d_row, col + d_col)
        if neighbor in graph:
            graph[(row, col)].append(neighbor)
```

2.3 Algoritmos aplicados

BFS usa una cola FIFO y explora por niveles. En un grafo no ponderado, garantiza encontrar la ruta con menor número de pasos. **DFS** usa una pila LIFO, profundiza por una rama antes de retroceder y puede encontrar una solución, pero no garantiza que sea la más corta. **A*** usa una cola de prioridad y combina el costo acumulado con una heurística. En este caso se usa distancia Manhattan, adecuada para movimientos verticales y horizontales.

La función **solve_maze()** integra el flujo: carga el laberinto, crea el grafo, calcula nodos y aristas, ejecuta cada algoritmo, genera las imágenes y almacena las métricas.

2.4 ¿Se puede establecer alguna métrica para evaluar los algoritmos?

Si. En este problema se pueden establecer metricas porque cada algoritmo produce una ruta, explora un conjunto de nodos y consume tiempo de ejecucion. Las metricas permiten evaluar efectividad, calidad de la solucion, esfuerzo computacional y consumo aproximado de memoria.

- **found:** indica si el algoritmo logro llegar desde la entrada hasta la salida.
- **path_steps:** cantidad de movimientos de la ruta final. Sirve para medir la calidad de la solucion.
- **visited_nodes:** cantidad de nodos visitados. Permite comparar cuanto explora cada algoritmo.
- **expanded_nodes:** nodos extraidos de la frontera y procesados. Refleja esfuerzo computacional.
- **max_frontier_size:** tamano maximo de la cola, pila o prioridad. Aproxima el uso de memoria.
- **time_ms:** tiempo de ejecucion en milisegundos. Permite comparar rapidez.
- **graph_nodes y graph_edges:** tamano del grafo generado a partir del laberinto.

Con estas metricas, BFS y A* se pueden evaluar por optimalidad cuando devuelven la menor cantidad de pasos. DFS se evalua principalmente por capacidad de encontrar una ruta y por cantidad de exploracion, pero no por optimalidad de ruta.

3. Metricas y comparacion de resultados

Laberinto	Alg.	Encontrado	Pasos	Visitados	Expandidos	Frontera max.	Tiempo ms
laberinto1	BFS	Si	14	43	43	5	0.025
laberinto1	DFS	Si	34	37	37	6	0.0426
laberinto1	A*	Si	14	36	36	5	0.0619
laberinto2	BFS	Si	44	183	183	13	0.072
laberinto2	DFS	Si	44	137	137	10	0.1318
laberinto2	A*	Si	44	140	140	15	0.2293
laberinto3	BFS	Si	344	859	859	10	0.4224
laberinto3	DFS	Si	378	1062	1062	157	0.8775
laberinto3	A*	Si	344	788	788	14	1.0278
laberinto4	BFS	Si	472	2410	2410	23	1.3138
laberinto4	DFS	Si	742	1238	1238	475	1.1892
laberinto4	A*	Si	472	2064	2064	27	5.5081

Interpretacion general: BFS encontro rutas optimas en numero de pasos para todos los laberintos, aunque puede visitar muchos nodos. DFS encontro soluciones, pero en los laberintos 1, 3 y 4 produjo rutas mas largas. A* tambien encontro rutas optimas en pasos y redujo nodos visitados en varios casos, aunque su costo de calculo puede aumentar por el uso de la cola de prioridad y la heuristica.

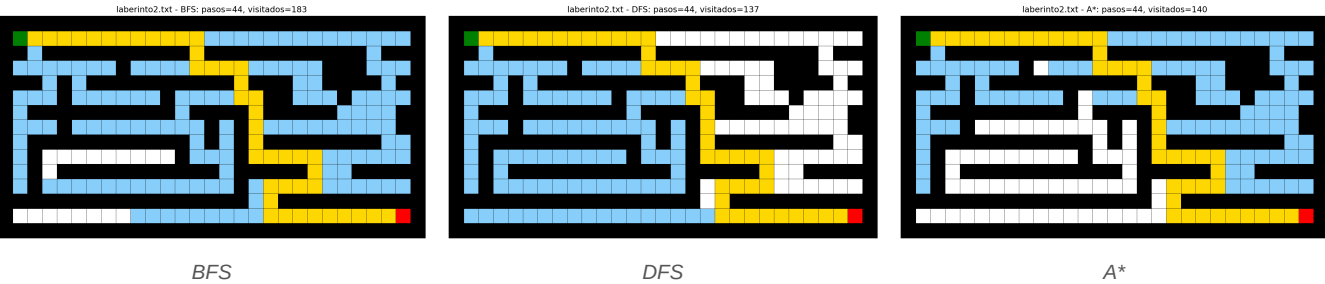
4. Imagenes de solucion de los laberintos

Las siguientes figuras corresponden a las imagenes generadas automaticamente en la carpeta **P1/results**. En cada caso se presenta la comparacion visual entre BFS, DFS y A*.

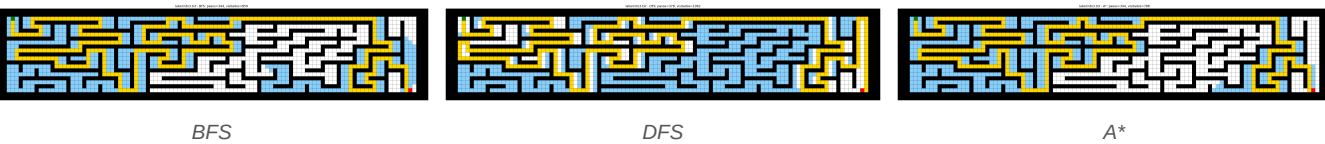
Laberinto 1



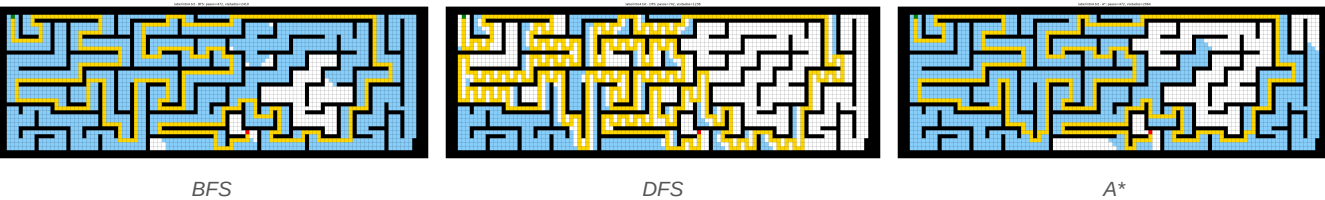
Laberinto 2



Laberinto 3



Laberinto 4



5. Parte 2 - Optimizacion por colonia de hormigas

5.1 Explicacion general del codigo ACO

La clase **AntColonyOptimization** modela una colonia de hormigas que busca una ruta desde un punto inicial hasta un punto final dentro de una grilla. El codigo define las posiciones de inicio y fin, obstaculos, tamano de grilla, numero de hormigas, tasa de evaporacion y pesos de feromona y heuristica. Cada hormiga construye un camino eligiendo vecinos con una probabilidad proporcional a la feromona disponible y a la cercania heuristica hacia la meta.

El metodo `_get_neighbors()` obtiene los movimientos validos sin atravesar obstaculos. El metodo `_select_next_position()` calcula la probabilidad de moverse a cada vecino usando la expresion basada en feromona y heuristica. Luego `find_best_path()` ejecuta varias iteraciones, construye caminos para todas las hormigas, evapora feromonas, refuerza la mejor ruta completa y actualiza la mejor solucion global.

5.2 A. Correr la implementacion planteada y analizar el codigo

En el caso de estudio 1, la implementacion se ejecuta con una grilla de 10 x 10, inicio en la coordenada (0, 0), salida en (4, 7) y obstaculos ubicados en la fila 2. El algoritmo genera rutas con varias hormigas, actualiza feromonas y finalmente grafica el mejor camino encontrado. La imagen resultante muestra la intensidad de feromona y la ruta reforzada como mejor solucion.

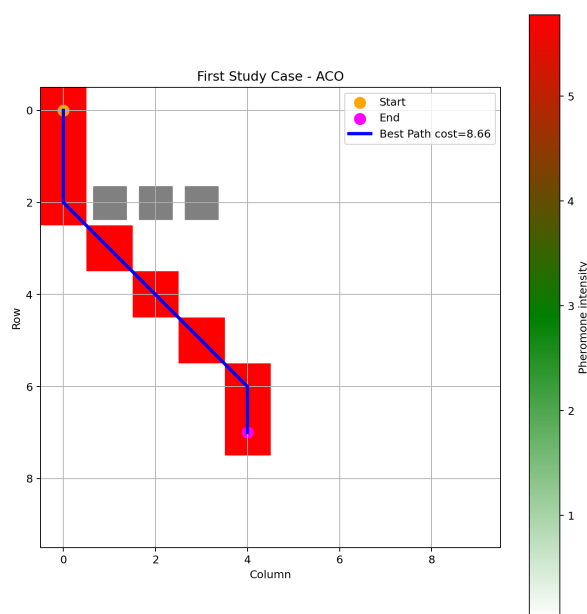


Figura 1. Resultado del caso de estudio 1 con ACO.

5.3 B. ¿Que ocurre con el segundo caso de estudio?

En el segundo caso de estudio se agrega un obstaculo adicional que modifica la ruta disponible. El problema original estaba en la seleccion del mejor camino: el codigo ordenaba todas las rutas por longitud y escogia la mas corta, aunque esa ruta podia estar incompleta o no llegar a la salida. Por tanto, no era suficiente elegir el camino con menor tamano.

La correccion consistio en agregar una condicion que filtre solo los caminos completos. Es decir, primero se verifica que el ultimo nodo del camino sea la meta. Despues, entre las rutas completas, se selecciona la de menor costo. Con esto se evita reforzar caminos cortos pero invalidos. Tambien se ajustaron parametros como numero de hormigas, evaporacion, alpha y beta para mejorar la probabilidad de convergencia.

Fragmento clave de la correccion del caso 2:

```
all_paths = [self._build_ant_path() for _ in range(self.num_ants)]
complete_paths = [path for path in all_paths if self._path_reaches_end(path)]

self._evaporate_pheromones()

if complete_paths:
    iteration_best = min(complete_paths, key=self._path_cost)
    self._deposit_pheromones(iteration_best)
    iteration_cost = self._path_cost(iteration_best)
    if iteration_cost < self.best_cost:
```

```
self.best_path = list(iteration_best)
self.best_cost = iteration_cost
```

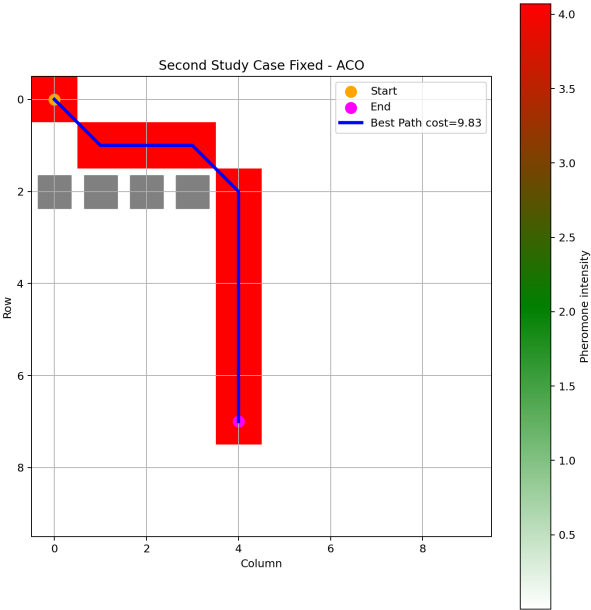


Figura 2. Segundo caso de estudio corregido.

5.4 C. Describir los parametros del modelo

Parametro	Proposito dentro del modelo
start	Coordenada inicial desde la que parten las hormigas.
end	Coordenada objetivo o fuente de recursos.
obstacles	Celdas bloqueadas que las hormigas no pueden atravesar.
grid_size	Tamano de la grilla sobre la cual se construyen los caminos.
num_ants	Cantidad de hormigas por iteracion. A mayor numero, mayor exploracion.
evaporation_rate	Porcentaje de feromona que se evapora en cada iteracion; evita quedarse prematuramente en rutas malas.
alpha	Peso de la feromona. Un valor alto favorece caminos ya reforzados.
beta	Peso de la heuristica. Un valor alto favorece movimientos mas cercanos a la meta.
pheromone_deposit	Cantidad base de feromona que se deposita en una ruta completa y valida.
iterations	Numero de ciclos de busqueda que realiza la colonia.
seed	Semilla aleatoria para reproducir los resultados.
allow_diagonal	Permite o restringe movimientos diagonales dentro de la grilla.

6. Random Search, Grid Search y aplicacion a TSP

6.1 D. ¿Que es Random Search y Grid Search? ¿Como aplicarlos?

Grid Search consiste en definir una grilla de valores posibles para cada hiperparametro y probar todas las combinaciones. Es ordenado y exhaustivo, pero puede volverse costoso cuando existen muchos parametros o rangos amplios. **Random Search** consiste en definir rangos o listas de valores y probar combinaciones aleatorias. No explora todo el espacio, pero suele ser mas eficiente cuando el algoritmo es estocastico y algunos parametros tienen mayor influencia que otros.

Para esta heuristica, la busqueda mas adecuada es **Random Search**, porque ACO tiene comportamiento aleatorio y parametros continuos como evaporation_rate, alpha y beta. En pocas ejecuciones puede explorar configuraciones diversas sin probar de forma exhaustiva todas las combinaciones.

En el codigo se implemento Random Search para el caso 2. En cada prueba se selecciona aleatoriamente el numero de hormigas, evaporacion, alpha, beta y semilla. Luego se ejecuta ACO y se registra si encontro ruta, el costo y el numero de nodos del camino. La mejor configuracion se escoge con base en menor costo entre rutas validas.

Trial	Encontrado	Costo	Nodos	Hormigas	Evap.	Alpha	Beta	Seed
1	Si	9.82843	10	20	0.1571	1.6534	2.3328	626
2	Si	9.82843	10	80	0.2645	0.9983	5.2604	2616
3	Si	9.82843	10	40	0.1849	1.0007	2.2258	28
4	Si	9.82843	10	80	0.3594	1.8179	2.888	109
5	Si	9.82843	10	60	0.3428	0.6531	1.2194	2330
6	Si	9.82843	10	40	0.366	0.5319	5.5558	9395
7	Si	9.82843	10	80	0.1561	1.7568	4.844	5628
8	Si	9.82843	10	100	0.3706	0.8103	4.0445	8636

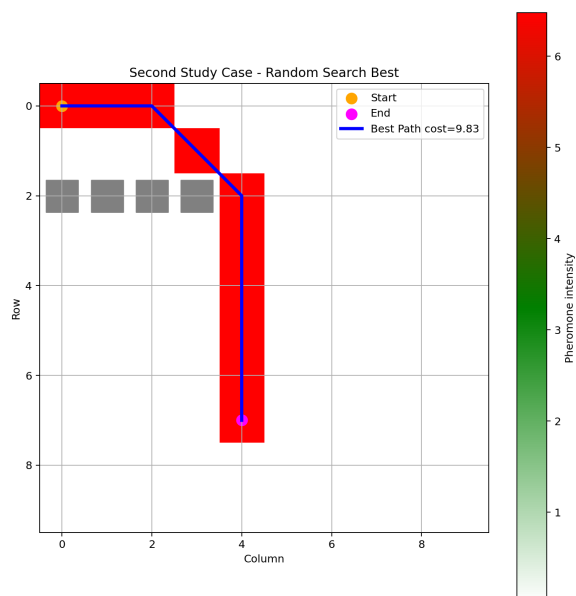


Figura 3. Mejor resultado del caso 2 mediante Random Search.

6.2 E. ¿Se puede utilizar ACO para resolver el Travelling Salesman Problem?

Si. ACO es una heurística adecuada para el Travelling Salesman Problem (TSP), porque el TSP consiste en encontrar una ruta de menor costo que visite un conjunto de ciudades una sola vez y regrese al origen. El comportamiento de las hormigas, basado en construcción de rutas, feromonas y preferencia por distancias cortas, se adapta naturalmente a este tipo de problema combinatorio.

Los pasos de implementación serían los siguientes:

- **Representar las ciudades como nodos** y calcular una matriz de distancias entre cada par de ciudades.
- **Inicializar una matriz de feromonas** para las conexiones entre ciudades.
- **Construir rutas con varias hormigas**, donde cada hormiga parte de una ciudad y selecciona la siguiente ciudad no visitada usando feromona y heurística de distancia inversa.
- **Completar el tour** hasta visitar todas las ciudades y regresar a la ciudad inicial.
- **Evaluar el costo** de cada tour como la suma total de distancias recorridas.
- **Evaporar feromonas** para reducir la influencia de rutas antiguas.
- **Depositar feromonas** con mayor intensidad en los tours más cortos.
- **Repetir varias iteraciones** y conservar el mejor tour global encontrado.

En TSP, a diferencia del laberinto, el objetivo no es llegar a una salida fija, sino construir un ciclo completo de menor distancia. Por ello, la condición de validez cambia: una solución válida debe visitar todas las ciudades exactamente una vez y retornar al nodo inicial.

7. Conclusiones

- La representación del laberinto como grafo se realizó en P1_MazeLoader.py mediante listas de adyacencia, usando como nodos las celdas transitables y como aristas los movimientos válidos en cuatro direcciones.
- BFS es el algoritmo más confiable para encontrar rutas mínimas en laberintos no ponderados; DFS encuentra rutas pero no garantiza optimalidad; A* combina costo y heurística para dirigir mejor la exploración.
- Las métricas path_steps, visited_nodes, expanded_nodes, max_frontier_size y time_ms permiten comparar calidad de ruta, esfuerzo de búsqueda, memoria aproximada y rapidez.
- En ACO, la selección del mejor camino debe considerar primero que el camino llegue a la meta. Escoger un camino solo por menor longitud puede reforzar rutas inválidas.
- Random Search fue apropiado para optimizar hiperparámetros del caso 2 porque el algoritmo es estocástico y los parámetros continuos no requieren una grilla exhaustiva para obtener una buena configuración.

- ACO puede aplicarse al TSP transformando las ciudades en nodos, las distancias en costos, y las rutas de las hormigas en tours completos que se refuerzan segun su longitud total.